

Universidad Carlos III de Madrid



Industrial Robotics group29 (2025/2026)

RAPID PROJECT REPORT

Group 1

Report done by:

Rosalía Carballo García

100495689

Guillermo Lucas Combarros

100495703

Rapid Project Report

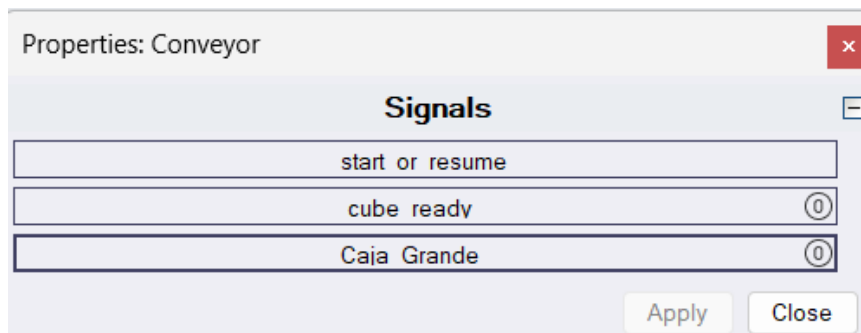
Introduction.....	3
Minimum Objectives	4
A.1. Describe the manufacturing process.....	4
A.2. Justification of the robot selection.....	5
A.3. TCPs usage.....	6
A.4. Definition of work objects	6
A.5. Capture by guidance of only one point	7
A.6. Approach points.....	7
A.7. Movement instructions.....	7
A.8. Not linear flow	7
A.9. Parameter passing routine	8
A.10. Justification of the use of robots	8
Extensions.....	9
B.1. More than one robot	9
B.3. FUNC routine	9
B.4. Additional RAPID functions	9
B.5. New Tools.....	10
B.6. TRAP routines	10
B.7. Optimization of procedures	11
B.8. Graphical Improvements	11
B.2. Functional summary	12

Introduction

In our factory, we have identified a significant bottleneck in the assembly of industrial metal boxes designed to produce high-security safes.

Performing the alignment and welding of hinges by hand takes a lot of time and often leads to structural inconsistencies which affect the final quality. To ensure precision and to scale the amount of production capacity we have decided to implement an automatic robotic cell using two specialized industrial robots. One of them is responsible for the handling and placement of the hinges, and the other robot executes the welding process. To eliminate human operators from the welding process is a security upgrade, as they are exposed to toxins. Also, for this reason and the fact that is a sophisticated task they need more shifts, making the process slower and increasing the cost. While welding with a robot in the long term is safer and cheaper.

Note: To start the simulation, it's necessary to press start to the in the conveyor.



Minimum Objectives

A.1. Describe the manufacturing process.

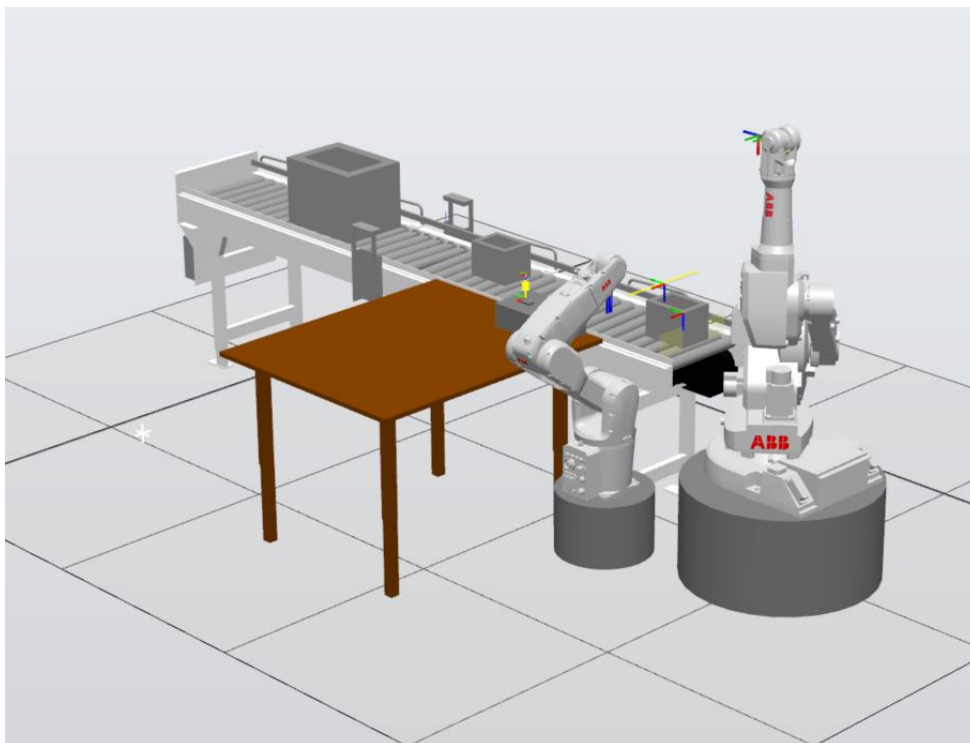
The chosen process is an automated assembly and welding of industrial boxes. This manufacturing process focuses on the automated attachment and reinforcement of metallic hinges onto industrial storage boxes of varying sizes (small and large).

It begins with a conveyor belt transporting the boxes into the workspace, mixing both sizes randomly. Two sensors placed at the end of the conveyor identify when the box is at the end and the size of it. The first robotic process, handled by the Handling Robot, using a specialized gripper, picks the hinges from a supply table and places them at precise coordinates on the box, selecting one for the small size and two for the large one.

Once the hinges are in position, it sends a digital signal to the welding robot. This robot performs a precise rectangular welding path around the hinge.

The coordinates of the location of the hinges and of the trajectory that the welding robot need to do are calculated relative to the box's WorkObject. That changes depending on the size.

Finally, the handling robot places the boxes on a side table, sorting them by size. Ideally, the process would include an additional conveyor to move the finished boxes to another part of the factory.

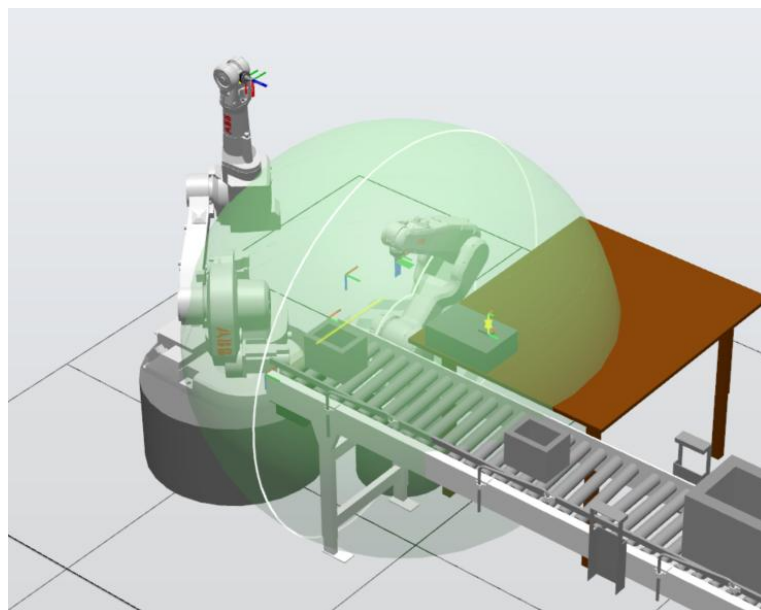


A.2. Justification of the robot selection.

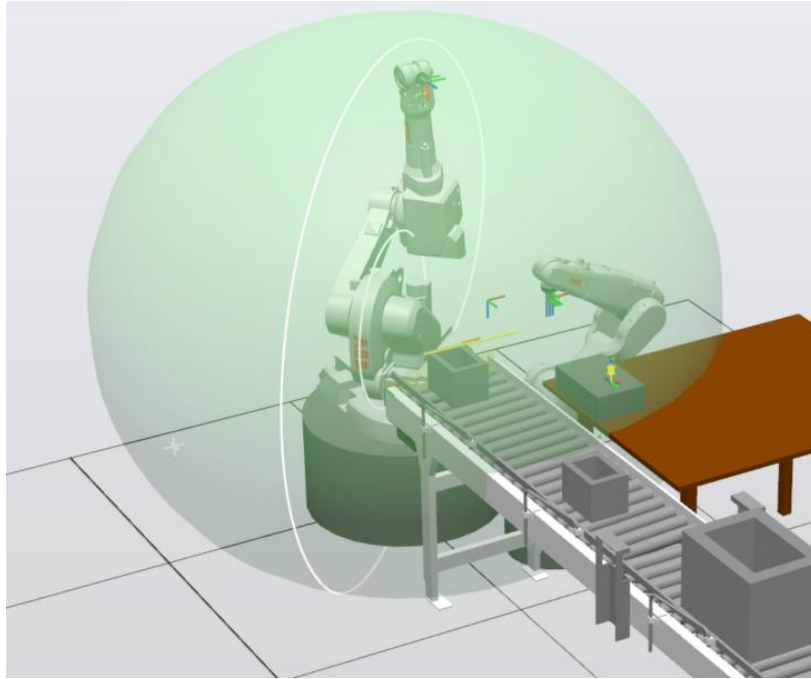
Our robotic cell uses two different ABB models to optimize the process:

- Handling Robot: ABB IRB 1200-5/0.9 (5 kg payload, 0.9 m reach).
This robot is used for the pick-and-place. The boxes, the hinge and the gripper tool weight around 3kg (in the worst case scenario). Since this robot supports up to 5 kg, we have a safety margin. With respect of the workspace, as it has 0.9 m reach, it allows it to perform these "pick and place" tasks easily, and fast as it is a compact robot.
- Welding robot: ABB IRB 1600-6/1.20 (6 kg payload, 1.2 m reach).
This robot only needs to support the weight of the welding torch, that is lighter than the 6 kg of payload. Furthermore, the larger workspace allows for better management of the bulky welding cable package, preventing tension during complex trajectories.

Both robots have 6 DOF, essential for our processes. As it needs to control both position and orientation of the tool. This means it is not redundant. To be redundant the robots should have more DOF.



Reach of handling robot



Reach of welding robot.

A.3. TCPs usage.

Two double-headed tools were utilized in this project.

The first set of tools are a gripper and a sensor with a TCP in one of the fingers of the gripper and in the middle. And the second tool is a welding torch and a vacuum cleaner to perform the suction of the gases produced at the welding.

A.4. Definition of work objects

We defined two additional work objects: “Wobj_cajag” and “Wobj_cajap”, to represent the specific coordinate systems for the large and small boxes. These are positioned at each box’s corner, ensuring that all trajectories (placement and welding paths) are recorded relative to the same framework. This makes the programming easily adjustable in the future: if the physical position of the boxes changes, we only need to update the work object coordinates instead of re-programming the movements.

A.5. Capture by guidance of only one point

We use only one reference point for each workobject: a “home” point for the robot when is not working and a corner point for each workobjects positioned at the corner of each box, large and small. Every other target in the manufacturing cycle was mathematically calculated using code transformation, specifically offsets functions. This method is the best that guarantees the trajectories to be consistent and relative to the workobjects. If in the future the shape of the boxes changes the trajectories can be easily adapted by only changing the reference point.

A.6. Approach points

We implemented approach points using offsets for all picking and welding tasks.

By using these precious points prevent accidental collisions with other boxes or the conveyor. Also, the robot can align its tool orientation and stabilize its trajectory before making physical contact, adding precision to the task.

A.7. Movement instructions

Two out of three available movement instructions were used to balance operational speed with path precision, as there were no applications for curved movement in our project. Joint movement is used for large-scale transitions between the Home position and the workspace, as it allows the robot to move at maximum speed along the most efficient trajectory where path geometry is not critical, for example when the robot is approaching the table in order to pick up the hinge. Linear movements were used for close approaches to targets, essential for avoiding collisions. For example when the Robot2 performs the welding path.

Furthermore, motion parameters are optimized by using “fine” for contact points to ensure exact positioning, while zoneZ is used for intermediate points to smooth the trajectory.

A.8. Not linear flow

The process carried out in the project is not linear since the boxes are created randomly and the conveyor detects the size of sheet that has arrived. This modifies the manufacturing sequence in real-time through the use of conditional statements and custom functions, for example the one that obtains the number of hinges each box needs. The cell automatically adapts its trajectories and tasks instead of following a fixed path.

A.9. Parameter passing routine

We have implemented a procedure that utilizes parameter passing. In particular, the procedure of picking up the boxes carried out by the first robot. In this procedure we use two parameters (a robtarget and a num), this adds modularity and reusability to our work. Passing the position of destiny makes it usable for different boxes at different heights or location without the need to write separate routines. The num parameter allows us to distinguish the type of box.

```
PROC Coger_Caja(robtarget pos_Destino,num tipo)
  MoveJ Offs(pos_Destino, 0, 0, -100), v100, z10, Pinza_Tool_agarre\WObj:=wobj_Activo;
  MoveL pos_Destino, v50, fine, Pinza_Tool_agarre \WObj:=wobj_Activo;
  SetDO do_ready,1;
  MoveL Offs(pos_Destino, 0, 0, -200), v50, fine, Pinza_Tool_agarre \WObj:=wobj_Activo;
  IF tipo=2 THEN
    MoveC Offs(Esquina,238.46,568.99,-50),Offs(Esquina,716.75,153.22,-50),v100,z50,Pinza_Tool_agarre\WObj:=wobj_Activo;

  ELSE
    MoveC Offs(Esquina,238.46,422,-50),Offs(Esquina,569.36,144.35,-50),v100,z50,Pinza_Tool_agarre\WObj:=wobj_Activo;
  ENDIF
  SetDO do_ready,0;
  MoveJ pHome,v300, fine, Pinza_Tool_agarre;
ENDPROC
```

A.10. Justification of the use of robots

The transition from manual or fixed to a robotic cell is justified by the three main criteria explained in “Implementation Criteria”: flexibility, quality, and safety.

First, robots ensure high-precision hinge placement and perfectly consistent welding beads. This precision and quality of the final product is impossible to replicate by hand in a large scale. Also, while fixed automation could perform these tasks it lacks the flexibility to adapt to different product variations; our robotic system can instantly switch between large and small boxes via software logic, whereas fixed machinery would require costly and time-consuming hardware retooling. Furthermore, carrying out the process by robots improves workplace safety by removing human operators from exposing to several times of exposure to the toxic fumes the welding process generates. Finally, the robotic solution optimizes cycle times making it the most cost-effective and reliable choice for this industrial process.

Extensions

B.1. More than one robot

We have added a second robot to implement the welding process. This second robot also uses two different TCPs other to “tool0” and two workobjects. The robots have real-time coordination through digital communication signals to manage the handoff between the handling and welding phases, making our factory more dynamic and complex.

B.3. FUNC routine

We implemented a custom function, which utilizes two distinct parameters, a “num” value for the hinge index and a “robotarget” as the coordinate reference for the box corner. It is used for the logic required to calculate precise offsets, allowing the robot to adapt the path for different box configurations (Small vs. Large) without duplicating code. By returning the calculated coordinates directly to the execution sequence.

```
FUNC robotarget ObtenerPosicionCaja(num indice_bisagra, robotarget esquina_ref)
  IF indice_bisagra = 2 THEN
    RETURN Offs(esquina_ref, -240, 18, -12);
  ELSE
    RETURN Offs(esquina_ref, -191.92, 23.66, 358.42);
  ENDIF
ENDFUNC
```

B.4. Additional RAPID functions

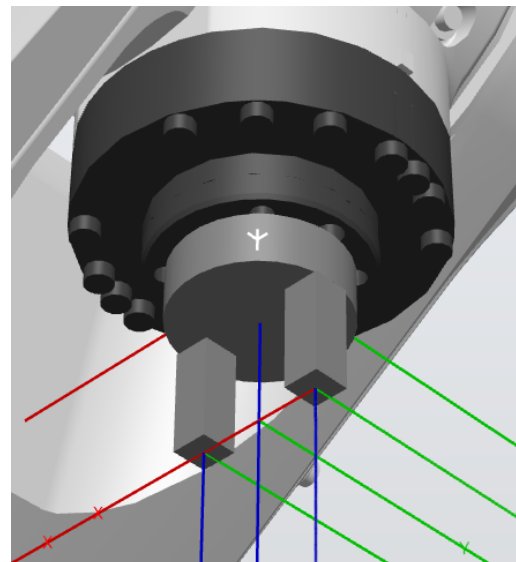
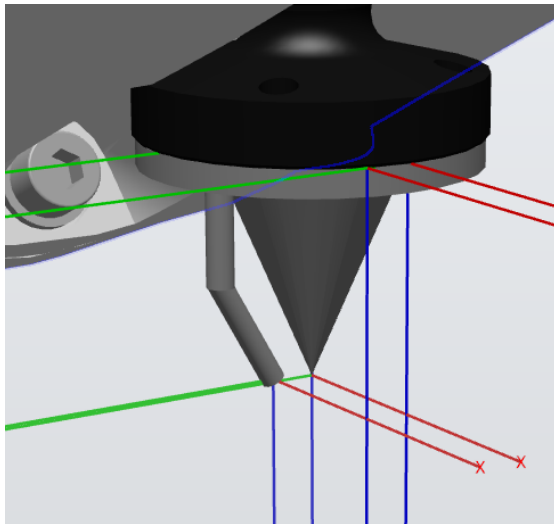
To improve the simulation and operational logic we incorporated several advanced RAPID instructions:

- SetDO and WaitDI: These were essential for the inter-robot communication protocol and synchronization. We used WaitDI to guarantee that the welding robot only starts its cycle when the handling robot is clear. SetDO was used to precisely control the activation of the vacuum gripper and the welding arc.
- MoveL/MoveJ Offs (Offset Transformation): By applying mathematical offsets to a reference target, we generated complex, multi-point welding paths and precise hinge placement coordinates without manually teaching every position, ensuring geometric consistency.

- TPWrite: To indicate that the emergency button has been pushed, we have used TPWrite, which sends a message to the console saying which robot has stopped.

B.5. New Tools

We needed specific tools, other than the ones provided in the laboratory sessions. For that reason, we created two different new tools. One tool is a hinge with a sensor at the top. It was designed in RobotStudio with basic shapes, to simulate a hinge but it cannot close and open itself. The other tool simulates the tip of a welding torch, additionally we added a small tube for the suction of gases produced during welding.



B.6. TRAP routines

In any industrial process using robots, it is essential to have an emergency stop button which once pressed, stops the robot's movement. To simulate this, we have created an input signal in the robot controller of the handling robot which in a real application would be the red mushroom. Once pressed (forcing the 1 of this variable), a TRAP routine will be executed, which will instantly stop the two robots. We thought that it would make more sense to use a single mushroom to stop both robots, and that's why we created an output for the first robot and connected it to an input of the second robot.

In the first robot, we initialize a digital input linked to a stopping function that, besides stopping its own movement, transmits a stop signal to the second robot. Once the second robot receives this signal, it also stops.

B.7. Optimization of procedures

The program is optimized by using specific variables like wobjdata. With wobjdata we have been able to simplify the instructions of MoveJ and MoveL since we moved with respect to that marked point and not with respect to the base of the robot. Using variables to switch between different box sizes in real-time, we created a modular system that is very easy to maintain.

B.8. Graphical Improvements

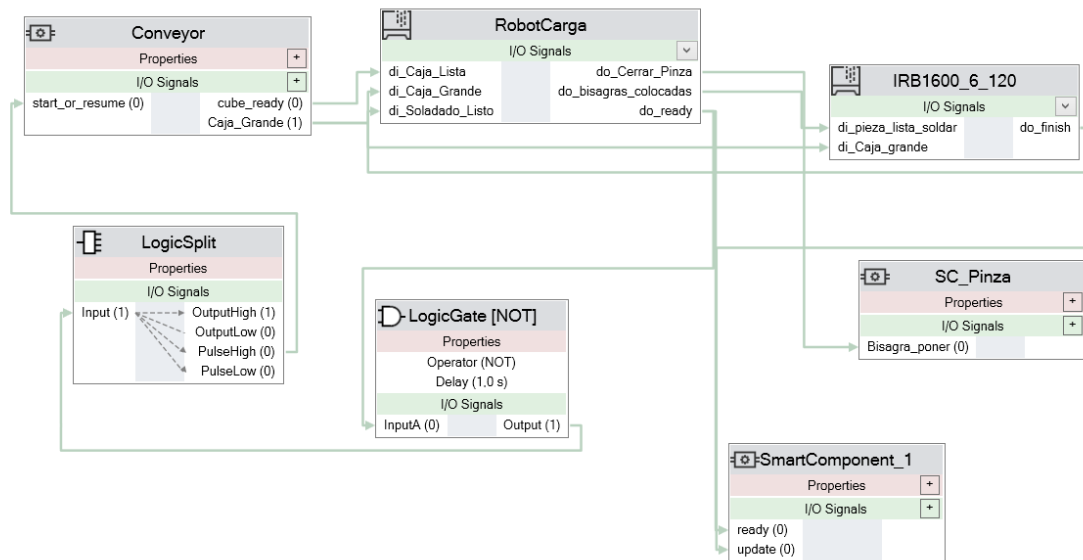
To improve the graphical aspect of the program and pursue realistic industrial flow, we integrated several smart components, in addition to the conveyor.

We used specific logic for the hinge component, utilizing "Source" and "Attacher" functions so that the robot's vacuum gripper can physically pick and place the part, transitioning it from a static supply target to a dynamic assembly state.

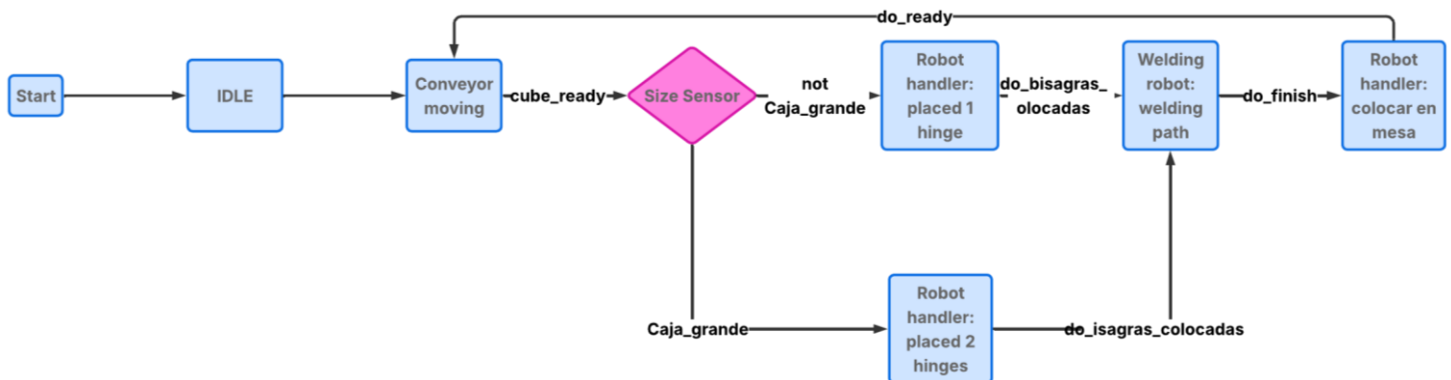
The most advanced optimization in our station is the custom box-hinge Union component. This smart component simulates the physical joining of the two parts once the assembly and welding process is completed. By connecting a digital signal to this component, we can trigger a "Show/Hide" and "Attacher" sequence that replaces the individual parts with a single, unified assembly.

B.2. Functional summary

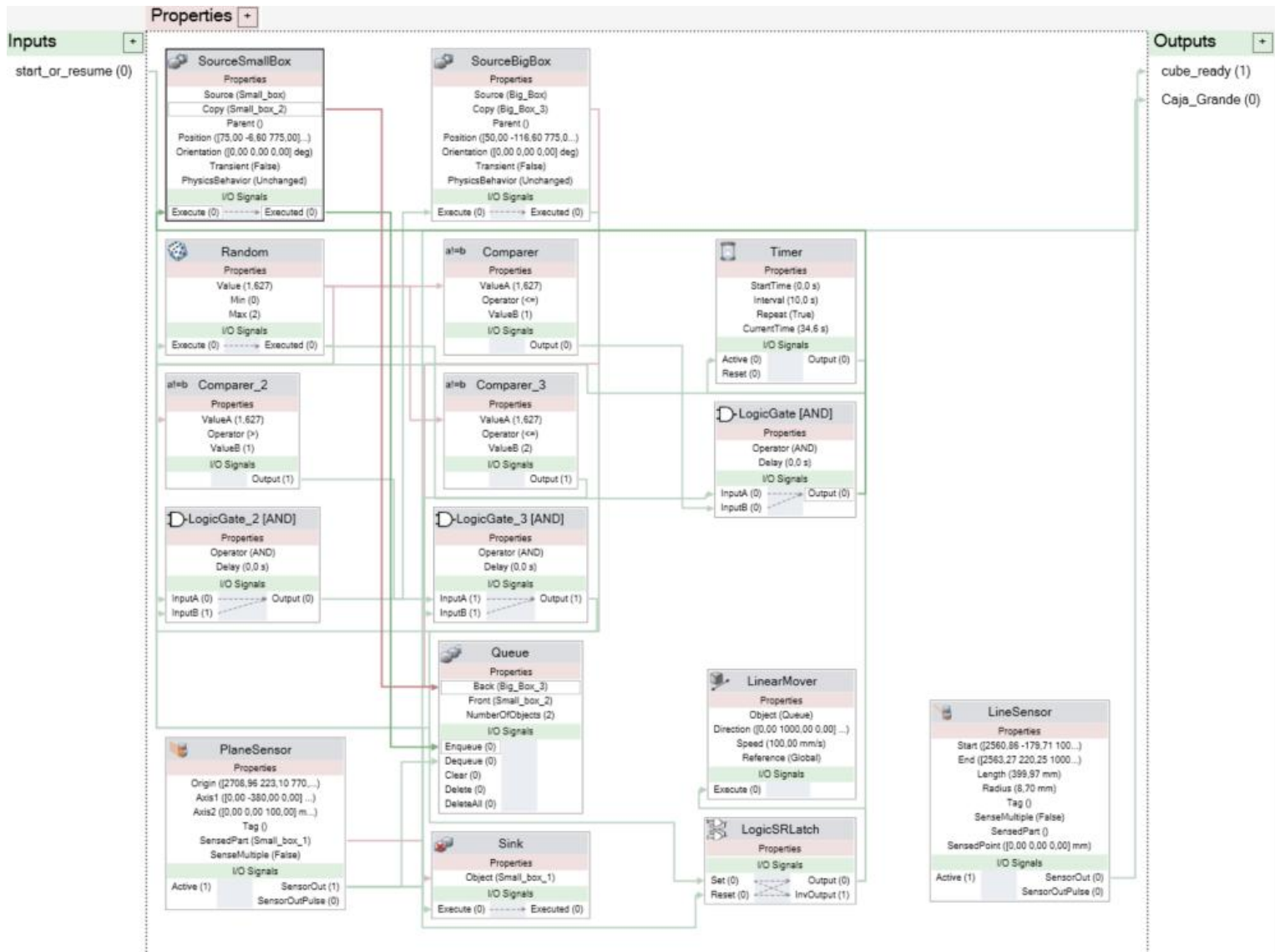
Station logic of the controllers and smart components:



Flow diagram of the process of a single box:



The production cycle begins at the Conveyor Belt, where a Source smart component generates the boxes randomly. As the box moves along the belt, it reaches a Position Sensor that identifies the arrival of the workpiece and triggers the “di_Caja_Lista” signal. Simultaneously, a line sensor determines whether the box is "Large" or "Small" through the “di_Caja_Grande”. This information is sent to the Handling Robot, which prepares its coordinate system (WorkObject) to match the dimensions of the incoming part.



Station Logic of the conveyor.

The IRB 1200 moves to the Hinge Supply Table. Using its gripper controlled by an Attacher/Detacher smart component the robot picks up a hinge. The robot then executes a the pick up and place cycle, positioning the hinges on the box. The number of hinges and their exact coordinates are calculated dynamically: the robot places two hinges for large boxes and one for small boxes, applying precise Offsets to ensure they are perfectly aligned with the box's edges.

```

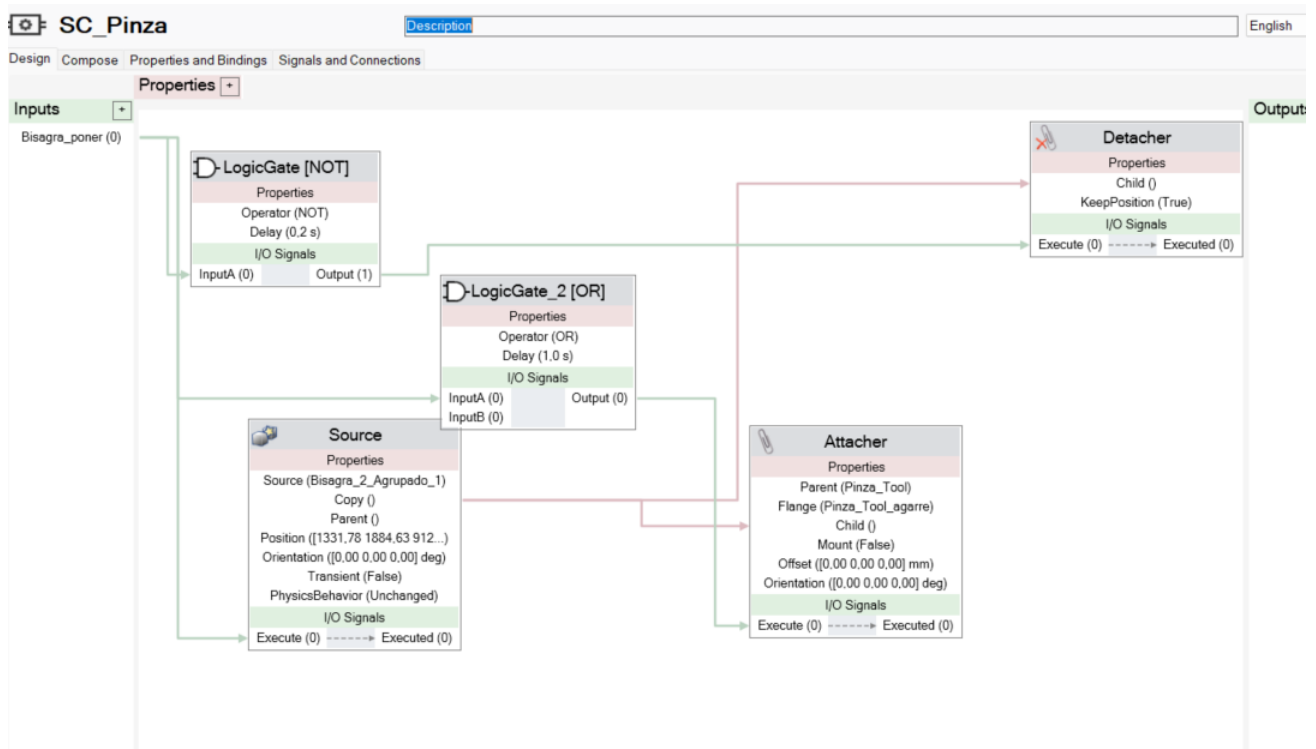
WHILE TRUE DO
    ! 1. wait
    WaitDI di_Caja_Lista, 1;
    ! 2. Configure work_object
    IF di_Caja_Grande=1 THEN
        wobj_Activo := Wobj_cajag;
    ELSE
        wobj_Activo := Wobj_cajap;
    ENDIF
    ! 3. Calculate hinges
    total_bisagras:=ObtenerCantidadBisagras();
    ! 4. For: movement
    FOR i FROM 1 TO total_bisagras DO
        Cogee_Bisagra;
        pos_Destino:=ObtenerPosicionCaja(i,Esquina); ! Measure where to leave the hinge
        MoveJ Offs(pos_Destino, 0, 0, -100), v300, fine, Pinza_Tool_agarre\WObj:=wobj_Activo;
        MoveL pos_Destino, v50, fine, Pinza_Tool_agarre \WObj:=wobj_Activo;
        Soltar_Bisagra;
        MoveL Offs(pos_Destino, 0, 0, -100), v300, fine, Pinza_Tool_agarre \WObj:=wobj_Activo;
    ENDFOR
    MoveJ pHome, v500, fine, Pinza_Tool_agarre;
    SetDO do_bisagras_colocadas, 1;
    WaitTime 1; ! Espera un segundo
    WaitDI di_Soladado_Listo,1;
    SetDO do_bisagras_colocadas, 0;
    Cogee_Caja pos_Destino,total_bisagras;
ENDWHILE

FUNC num ObtenerCantidadBisagras()
    IF di_Caja_Grande = 1 THEN
        RETURN 2;
    ELSE
        RETURN 1;
    ENDIF
ENDFUNC

FUNC robtarget ObtenerPosicionCaja(num indice_bisagra, robtarget esquina_ref)
    IF indice_bisagra = 2 THEN
        RETURN Offs(esquina_ref, -240, 20, -9.5);
    ELSE
        RETURN Offs(esquina_ref, -91,20,-9.5);
    ENDIF
ENDFUNC

```

While loop of Handling Robot with (hint placement logic).

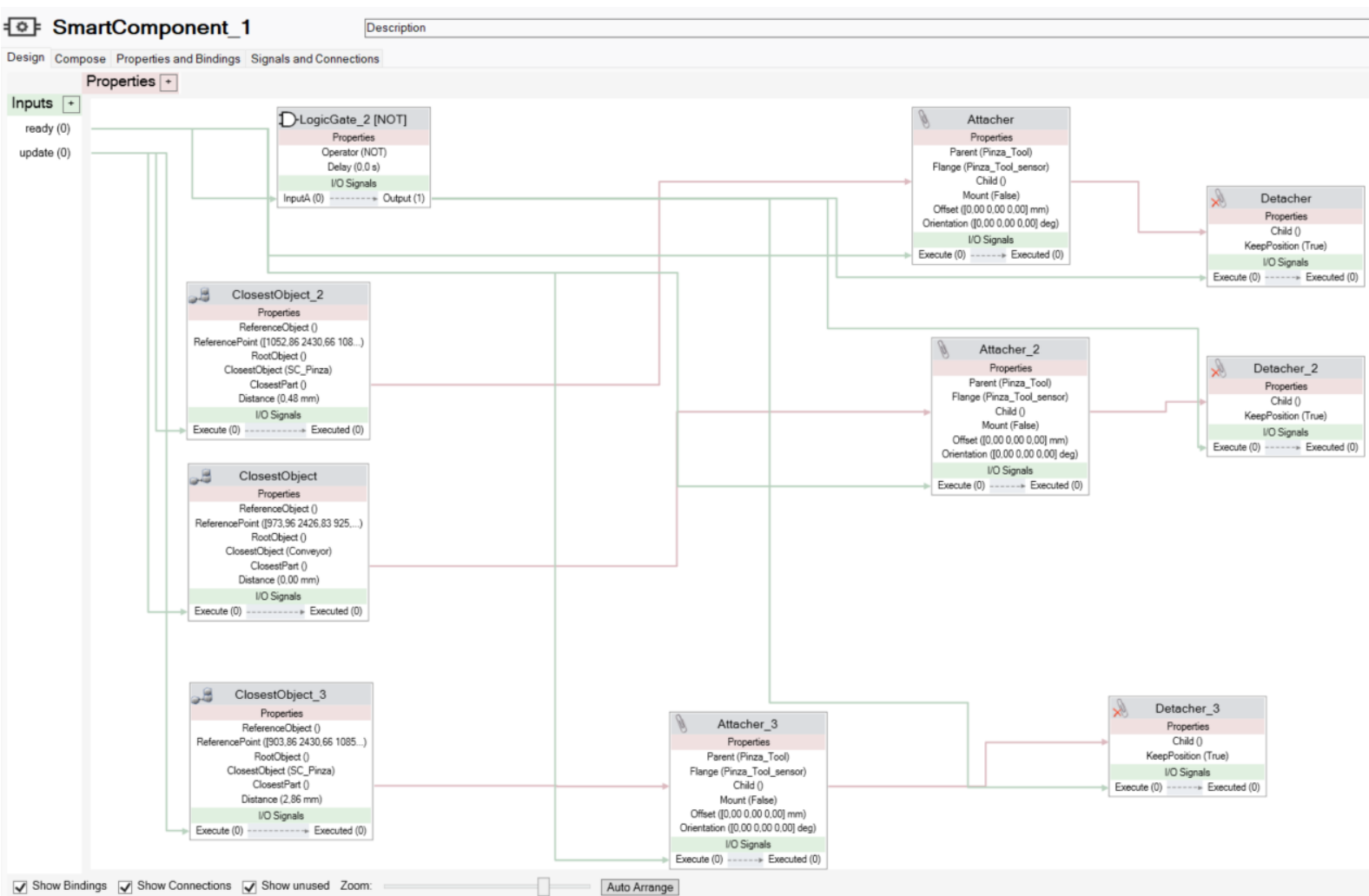


Smart Component for the hinge.

After the hinges are placed, the Handling Robot sends a synchronization signal (do_bisagras_colocadas) to the Welding Robot. At this stage, the first robot enters a waiting state while the welding robot takes over. The robot executes a welding path along the joints of the hinges. During this interaction, we use a custom-made Box-Hinge Union smart component. This component is triggered at the end of the welding process to "merge" the individual parts into a single, unified assembly, simulating a real industrial permanent union.

```
PROC Soldar_Contorno(robtarget inicio)
  MoveJ RelTool(inicio,50,0,-50,\Ry:=-35),v500,fine,Soldar_Tip\WObj:=wobj_Activo;
  MoveLDO RelTool(inicio,0,0,0,\Ry:=-35),v50,fine,Soldar_Tip\WObj:=wobj_Activo,do_soldando,1;
  MoveLDO RelTool(inicio,0,24,0,\Ry:=-35),v50,fine,Soldar_Tip\WObj:=wobj_Activo,do_soldando,0;
  MoveL RelTool(inicio,50,24,-50,\Ry:=-35),v50,fine,Soldar_Tip\WObj:=wobj_Activo;
  MoveJ RelTool(inicio,0,74,-50,\Rx:=35),v500,fine,Soldar_Tip\WObj:=wobj_Activo;
  MoveLDO RelTool(inicio,0,24,0,\Rx:=35),v50,fine,Soldar_Tip\WObj:=wobj_Activo,do_soldando,1;
  MoveLDO RelTool(inicio,-30,24,0,\Rx:=35),v50,fine,Soldar_Tip\WObj:=wobj_Activo,do_soldando,0;
  MoveL RelTool(inicio,0,74,-50,\Rx:=35),v50,fine,Soldar_Tip\WObj:=wobj_Activo;
  MoveJ RelTool(inicio,-80,24,-50,\Ry:=45),v500,fine,Soldar_Tip\WObj:=wobj_Activo;
  MoveLDO RelTool(inicio,-30,24,0,\Ry:=45),v50,fine,Soldar_Tip\WObj:=wobj_Activo,do_soldando,1;
  MoveLDO RelTool(inicio,-30,0,0,\Ry:=45),v50,fine,Soldar_Tip\WObj:=wobj_Activo,do_soldando,0;
  MoveL RelTool(inicio,-80,24,-50,\Ry:=45),v500,fine,Soldar_Tip\WObj:=wobj_Activo;
ENDPROC
```

Procedure for the welding.



Station logic of the merge between the boxes and the hinges.

When the welding is finished, the IRB 1600 sends a feedback signal (di_Soldado_Listo) back to the Handling Robot. The handling robot then reactivates to perform the unloading stage. It grips the complete box and lifts it from the workstation. To ensure a smooth and collision-free transfer to the storage area, the robot follows a curved trajectory implemented with MoveC (Circular Motion) instructions.

```

PROC Coger_Caja(robtarget pos_Destino,num tipo)
MoveJ Offs(pos_Destino, 0, 0, -100), v100, z10, Pinza_Tool_agarre\WObj:=wobj_Activo;
MoveL pos_Destino, v50, fine, Pinza_Tool_agarre \WObj:=wobj_Activo;
SetDO do_ready,1;
MoveL Offs(pos_Destino,0,0,-10), v50, fine, Pinza_Tool_agarre \WObj:=wobj_Activo;
MoveJ Offs(pos_Destino,0,0,-10), v50, fine, Pinza_Tool_agarre \WObj:=Wobj_cajag;
IF tipo=2 THEN
MoveC Offs(Esquina,-126.05,402.87,-150),Offs(Esquina,683.05,274.72,-150),v100,fine,Pinza_Tool_agarre\WObj:=Wobj_cajag;
MoveL Offs(Esquina,683.05,274.72,-50),v100,fine,Pinza_Tool_agarre\WObj:=Wobj_cajag;
MoveJ Offs(Esquina, 515.71,590.53,-107.24), v100, z10, Pinza_Tool_agarre\WObj:=wobj_Activo;
ELSE
MoveC Offs(Esquina,238.46,422,-200),Offs(Esquina,501.51,491.07,-200),v100,z50,Pinza_Tool_agarre\WObj:=Wobj_cajag;
MoveL Offs(Esquina,501.51,491.07,100),v100,fine,Pinza_Tool_agarre\WObj:=Wobj_cajag;
ENDIF
SetDO do_ready,0;
MoveJ pHome,v300, fine, Pinza_Tool_agarre;
ENDPROC

```

Procedure for storing the boxes.

Finally, the completed assembly is placed on the Storage Table. Once the box is released, the robot returns to its Home position, and the system resets, ready to detect the next part on the conveyor and repeat the cycle.

Throughout the entire operation, the system is protected by a safety routine. A Red Mushroom button is always available. If pressed, it triggers an immediate StopMove through a RAPID interruption (TRAP), halting both robots instantly and displaying an emergency message on the operator's screen to ensure human and machine safety.

```

]TRAP Trap_Emergencia
    StopMove;
    SetDO do_stop_robot2, 1;
    TPWrite "EMERGENCY: Handling robot stopped";
ENDTRAP

```

Trap of the handling robot.

```

]TRAP Trap_Emergencia
    StopMove;
    SetDO do_Soldando, 0; ! turns out the welding arch
    TPWrite "EMERGENCY: Welding robot stopped";
ENDTRAP

```

Trap of the welding robot.